# Simple Folder Backup Tool

```
import os
import shutil
import argparse
import logging
import zipfile
from datetime import datetime

logging.basicConfig(
    filename="backup.log",
    level=logging.INFO,
    format="%(asctime)s - %(message)s"
)


def backup(source, destination, dry_run, make_zip, keep):

    if not os.path.isdir(source):
        print("Source folder not found!")
        return

    os.makedirs(destination, exist_ok=True)

    # Find previous backup
    backups = sorted([
        x for x in os.listdir(destination)
        if x.startswith("backup_")
        and os.path.isdir(os.path.join(destination, x))
    ])

    previous = (
        os.path.join(destination, backups[-1])
        if backups else None
    )

    # Create timestamped backup
    name = "backup_" + datetime.now().strftime("%Y%m%d_%H%M%S")
    new_backup = os.path.join(destination, name)

    print("Creating:", name)
    logging.info("Backup started")

    if not dry_run:
        os.makedirs(new_backup)

    # Copy new and changed files
    for root, dirs, files in os.walk(source):

        for file in files:

            source_file = os.path.join(root, file)
            relative = os.path.relpath(source_file, source)

            old_file = (
                os.path.join(previous, relative)
                if previous else None
            )

            changed = (
                old_file is None
                or not os.path.exists(old_file)
                or os.path.getmtime(source_file)
                   > os.path.getmtime(old_file)
            )

            if changed:
                print("Copying:", relative)
                logging.info("Copied: " + relative)

                if not dry_run:
                    destination_file = os.path.join(
                        new_backup, relative
                    )

                    os.makedirs(
                        os.path.dirname(destination_file),
                        exist_ok=True
                    )
                    shutil.copy2(
```

```
                    source_file,
                    destination_file
                )

    # Copy unchanged files from previous backup
    if previous and not dry_run:

        for root, dirs, files in os.walk(previous):

            for file in files:

                old_file = os.path.join(root, file)
                relative = os.path.relpath(old_file, previous)

                new_file = os.path.join(
                    new_backup, relative
                )

                if not os.path.exists(new_file):
                    os.makedirs(
                        os.path.dirname(new_file),
                        exist_ok=True
                    )

                    shutil.copy2(old_file, new_file)

    # Create ZIP if requested
    if make_zip and not dry_run:

        zip_file = new_backup + ".zip"

        with zipfile.ZipFile(
            zip_file,
            "w",
            zipfile.ZIP_DEFLATED
        ) as z:

            for root, dirs, files in os.walk(new_backup):

                for file in files:

                    path = os.path.join(root, file)

                    z.write(
                        path,
                        os.path.relpath(path, new_backup)
                    )

        shutil.rmtree(new_backup)

        print("ZIP created:", zip_file)
        logging.info("ZIP created")

    # Rotate old backups
    all_backups = sorted([
        x for x in os.listdir(destination)
        if x.startswith("backup_")
    ])

    for old in all_backups[:-keep]:

        path = os.path.join(destination, old)

        print("Removing old backup:", old)
        logging.info("Removed old backup: " + old)

        if not dry_run:

            if os.path.isdir(path):
                shutil.rmtree(path)
            else:
                os.remove(path)

    print("Backup completed!")
    logging.info("Backup completed")


# Command line arguments

parser = argparse.ArgumentParser(
    description="Simple Folder Backup Tool"
)
```

```
parser.add_argument(
    "--source",
    required=True,
    help="Source folder"
)

parser.add_argument(
    "--destination",
    required=True,
    help="Backup destination"
)

parser.add_argument(
    "--dry-run",
    action="store_true",
    help="Show changes without copying"
)

parser.add_argument(
    "--zip",
    action="store_true",
    help="Compress backup into ZIP"
)

parser.add_argument(
    "--keep",
    type=int,
    default=3,
    help="Number of backups to keep"
)

args = parser.parse_args()

backup(
    args.source,
    args.destination,
    args.dry_run,
    args.zip,
    args.keep
)
```